

LEAD Quiz Portal — Feature & Implementation Brief

Proctored online exam platform · 450 concurrent students · 5 proctor stations · identity = roll number

Stack: Next.js 15 + TypeScript (Vercel) · Supabase Postgres + RLS + Storage + Auth · Upstash Redis (locks, rate limits) · YOLO11n via ONNX Runtime Web (client-side) · pg_cron (timers, sweeps). **Alternative:** single VPS with Docker Compose — NestJS + Socket.IO + BullMQ + MinIO.

Core features

Feature	How to implement	Critical gotcha
1 Auth & ID verification Roll number + issued password	- Bulk-import roster; generate 8-char passwords, argon2id hashed. Export once to printed slips. - Supabase Auth with email <code>{roll}@college.internal</code> → free JWT that RLS understands. - Single-session lock: Redis <code>session:{id}</code> holding device fingerprint; second device refused. - Preflight gate: camera + mic + screen-share + fullscreen + single-display + face match (InsightFace).	Never auto-block on face mismatch — flag for review. Lockout must be admin-clearable in one click; you will need it.
2 Answer secrecy Students cannot find the key	- Answer key in its own table/schema with no RLS policy for students — deny is the default. - Student question DTO selects <code>id, body</code> only. Never <code>include</code>, never <code>spread</code>. - Grade server-side <i>after</i> submit. No "check answer" route exists during the exam. - Per-student option shuffle seeded by attempt id, so "answer is B" is useless to a friend.	Client-side obfuscation is worthless — anything shipped to the browser is readable. Secrecy comes only from never grading in the browser.
3 Fullscreen lock & focus monitoring 3 flags → auto-submit + logout	- Listen: <code>fullscreenchange</code>, <code>visibilitychange</code>, <code>blur</code> (2s grace), blocked shortcuts, devtools probes. - Server counts flags — never the client. Flags 1–2: overlay + evidence capture + "flag N of 3". - Flag 3: submit (don't discard), revoke session, denylist the JWT <code>jti</code>, tear down media tracks. - RLS policy rejecting writes to any attempt not <code>IN_PROGRESS</code> is the real enforcement.	Alt-Tab fires <code>blur</code> but not <code>fullscreenchange</code> — listen for both. Re-entering fullscreen needs a click, so the overlay must have a button.
4 Camera proctoring (YOLO) Phone / multiple people / identity	- YOLO11n in-browser via ONNX Runtime Web, 1 frame / 3s. Upload only "interesting" frames + 1 baseline / 30s. - Smooth before acting: require N-of-M frames to agree, then cool down 60–120s. - Optional server second opinion on queued items only — Modal/Replicate serverless GPU, a few dollars.	A water bottle reads as a phone. Detections must never auto-flag — see feature 5.
5 Human review queue ML proposes, admin decides	- A detection creates an event with <code>status: PENDING_REVIEW</code>, <code>appliedWeight: 0</code> — zero effect on the student. - Admin card shows webcam shot + screen shot + ± 3s frame strip. Keys: A approve, R record-only, D dismiss. - Throttle or drown: max 3 pending per student, confidence floor rises with queue depth, 15-min auto-expiry. - Every dismissal is a labelled false positive → live precision-per-detector tile.	Without the throttles the queue hits thousands, proctors rubber-stamp, and you're back to auto-flagging with worse accuracy.
6 Screen capture Periodic screenshots	<code>getDisplayMedia({displaySurface:'monitor'})</code>; reject anything that isn't a full monitor. <code>ImageCapture.grabFrame()</code> every 20s → WebP q0.5 (~60 KB). Re-verify surface on every heartbeat. - Full video recording only for high-risk students, toggled remotely (77 GB vs 7 GB per exam).	You cannot screenshot silently — the permission prompt and sharing indicator are unavoidable. Design around it.
7 Microphone monitoring Speech during a silent exam	- AudioWorklet computes RMS + zero-crossing rate locally; upload nothing unless it fires. - On trigger, send the last 10s from a rolling buffer → WebRTC VAD + speaker count server-side. - Calibrate the baseline from 5s of silence at preflight.	Do not transcribe — privacy and legal problem. Weight audio low; it ranks who to look at, never blocks.

Feature	How to implement	Critical gotcha
8 Evidence pipeline Snapshot pair on every anomaly	- On any anomaly capture webcam + screen together, share one <code>clientEventId</code>, POST with <code>keepalive</code>. - Browser uploads direct to Storage via signed URL — never through your API. Store under roll number. - Risk score = Σ weights with 25-min decay, counting only confirmed events. - On block, freeze a case file: report PDF + events.jsonl + media + sha256 manifest.	Routing media through serverless functions is the big cost trap: invocations plus bandwidth for no benefit.
9 Anti-screenshot Deterrence + attribution	- Per-student tiled watermark (roll + name + clock), two layers; MutationObserver fires CRITICAL if removed. - Blur shield on window blur / PrintScreen; clear clipboard on PrintScreen. - Enforce single display at preflight. Question pools + shuffle make any leak low-value.	You cannot prevent screenshots in a browser. Only an Electron client with <code>setContentProtection</code> comes close — and nothing stops a phone camera.
10 Issue flagging & admin chat 1:1 by roll number	- Always-visible "Report an issue" button → category + description + auto screenshot → opens a thread. - Atomic claim via Redis <code>SET NX</code> so two of five stations never answer the same student. - Admin actions on the same channel: grant time, clear warnings, unblock, force submit, capture now. - Canned replies and a broadcast box — you will type the same six answers all day.	Keep students off Supabase Realtime (500-connection cap). Piggyback server→client messages on the 5s heartbeat response; only admins and active chats need sockets.
11 Admin dashboard Live grid + unblock console	- Virtualized 450-tile grid sorted by risk descending, lazy 96×72 thumbnails, one aggregate push per 5s. - Never 450 live streams — open WebRTC only when a tile is clicked. <code>/admin/unblock</code>: search a roll number → see the 3 flags with evidence → mandatory reason → restore. - Defaults: warnings back to 2 (not 0), lockout minutes auto-credited. Bulk unblock for outages.	Automatic termination is only defensible because a human can reverse it in seconds. Build this before you enable feature 3.
12 Progress checkpointing No unfair disadvantage	- Upsert every answer on click with a <code>revision</code> guard so a stale retry can't overwrite newer data. - IndexedDB queue when offline + a visible "saved 2s ago" pill. <code>deadlineAt = startedAt + duration + pausedMs</code>. Cron opens an interruption when heartbeats stop. - If $\geq 15\%$ of the cohort drops in one minute it's <i>your</i> outage → credit time automatically to all affected.	One student dropping is their Wi-Fi — send it to a proctor. Auto-crediting individual drops is trivially farmed.

Build order: scaffold → auth → exam core + checkpointing → focus proctoring → media capture → chat → admin dashboard + unblock → ML + review queue → anti-leak → hardening. **≈52 dev-days.**

Non-negotiable before exam day: a 450-VU load test, and a 50-student mock exam on identical infrastructure a week out — including the failure drills (kill Wi-Fi, kill the API, terminate then unblock a student).

Consent & law: explicit timestamped consent listing what is captured and for how long; 30-day retention for non-flagged media; every sanction traceable to a named human (DPDP Act 2023).